

aubergine.®

Designing Reliable Connected- Device Apps with Swift & BLE

14 MAR 2026

*Confidential document. Not to be shared without consent.

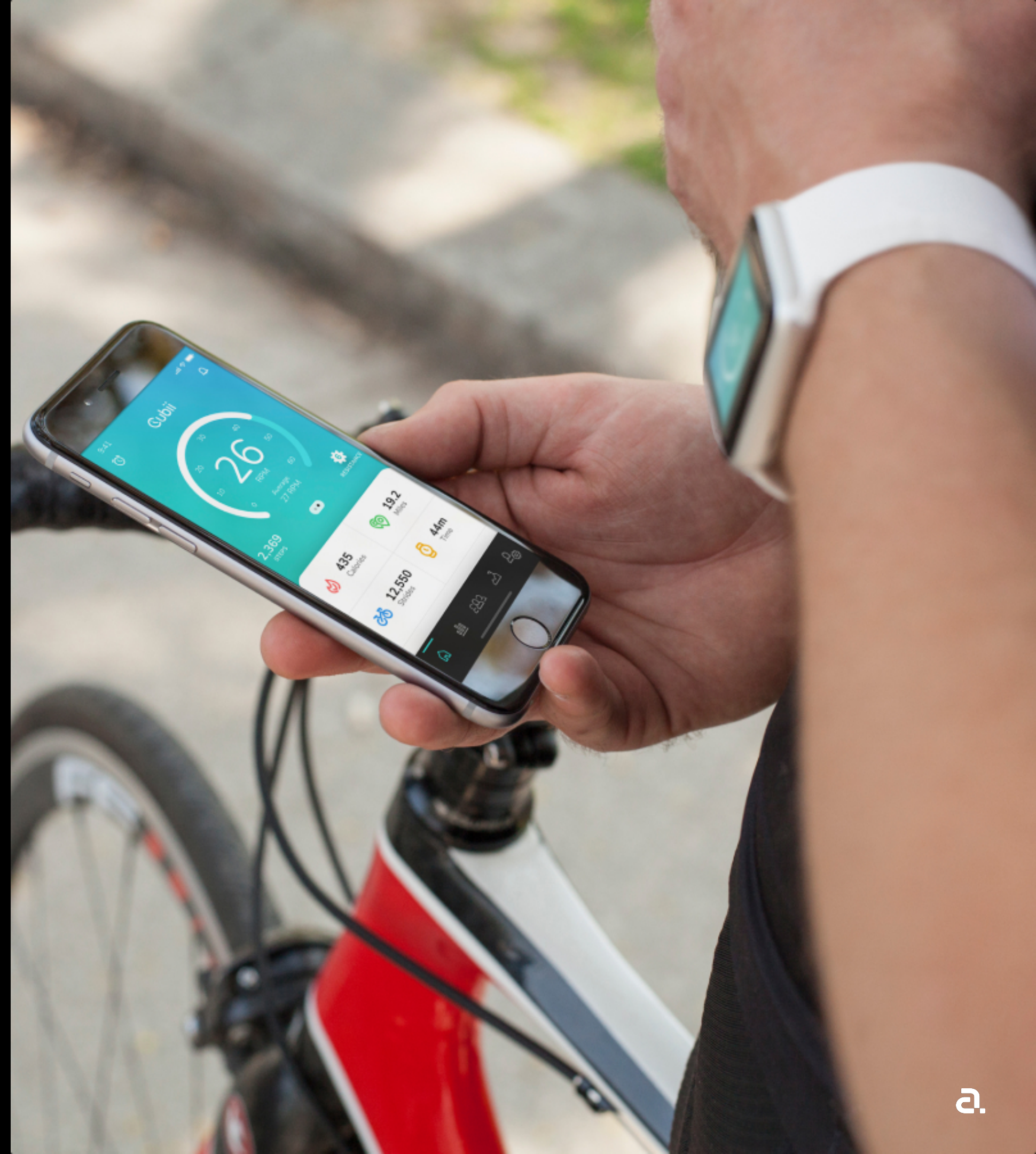


THE RISE OF CONNECTED DEVICES

Mobile apps are the interface to physical devices

Bluetooth devices are everywhere:

- Wearables and health devices
- Smart home accessories
- Fitness trackers



WHY BLE APPS ARE CHALLENGING

BLE demos are easy.
Production BLE apps are
not.

Challenges:

- Unstable wireless connections
- Limited bandwidth
- iOS background restrictions
- Firmware inconsistencies
- Battery constraints



WHAT WE'LL COVER

We'll move from BLE fundamentals to building production-ready BLE apps.

Content:

- BLE fundamentals
- BLE development in Swift
- Reliable BLE architecture
- Real-world reliability challenges
- iOS background modes & permissions
- Testing and debugging tools
- Live App Demo
- Production Lessons



01 BLE FUNDAMENTALS

WHAT IS BLUETOOTH LOW ENERGY?

BLE trades bandwidth for extremely low power consumption.

Bluetooth Low Energy (BLE) is designed for:

- Low power consumption
- Small data packets
- Intermittent communication
- Battery-powered devices

BLE ROLES

Most mobile apps act as **centrals** connecting to device **peripherals**.

BLE communication involves two roles:

- Central → Phone
- Peripheral → Device



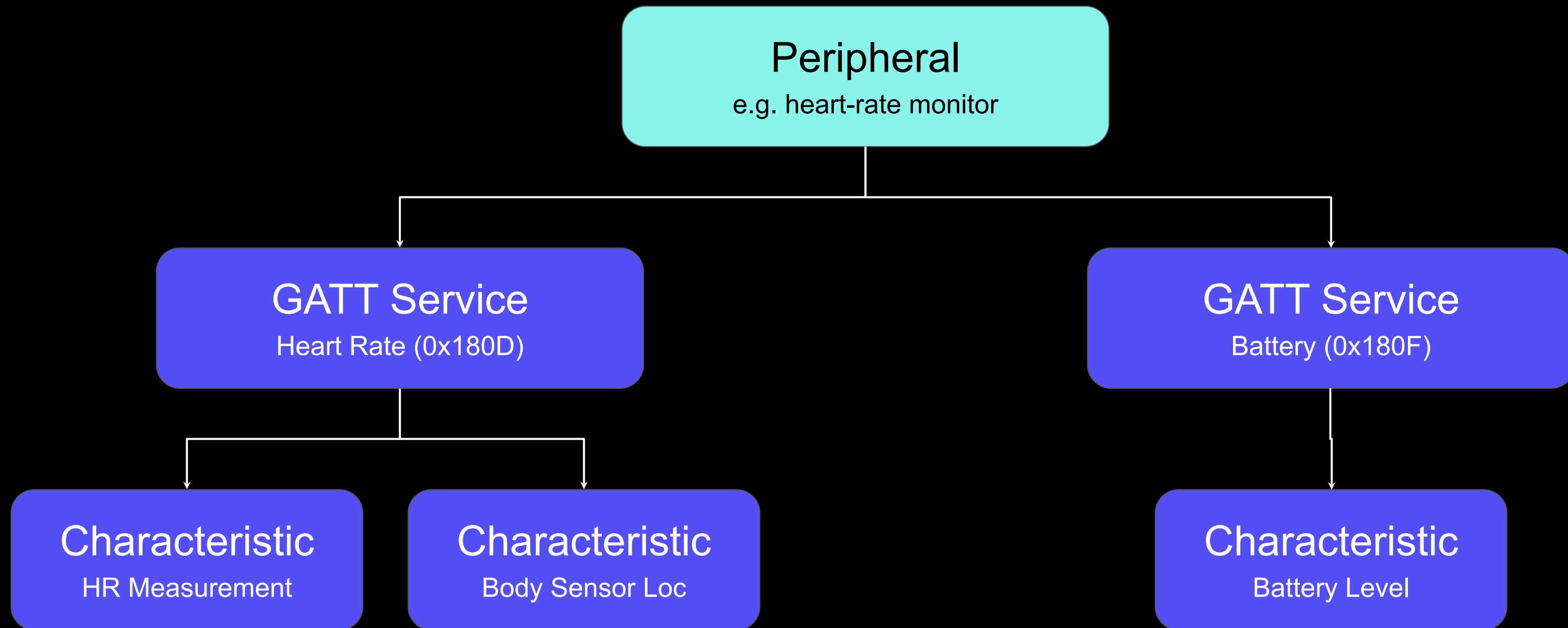
BLE DATA MODEL (GATT)

BLE devices expose functionality through **services** and **characteristics**.

BLE data is organized using GATT.

- Services
- Characteristics
- Descriptor

BLE DATA MODEL (GATT)



Properties:

Read

Write

Notify

Indicate

BLE communication
follows a strict lifecycle.

Typical BLE interaction:

- Scan for nearby devices
- Discover peripheral
- Connect to device
- Discover services
- Discover characteristics
- Read / Write data
- Subscribe for notifications



02 BLE DEVELOPMENT IN SWIFT

/// CORE BLUETOOTH FRAMEWORK

CoreBluetooth provides the foundation for BLE communication on iOS.

Key classes:

CBCentralManager

Manages scanning and connections

CBService

Represents device services

CBPeripheral

Represents the device

CBCharacteristic

Represents device data points

BEGINNER MISTAKE

Mixing UI and BLE logic leads to fragile apps.

Common mistake:

- Putting BLE logic inside UI controllers.

Problems:

- Tight coupling
- Difficult debugging
- Poor testability
- Hard to scale

Solution:

- Separate BLE communication from UI logic.



03 RELIABLE BLE ARCHITECTURE

RECOMMENDED ARCHITECTURE

```
BLE/  
├── Protocol/  
│   ├── BlePacket.swift  
│   └── BleDeviceProtocol.swift  
├── Transport/  
│   └── BleTransport.swift  
├── Manager/  
│   ├── BleManager.swift  
│   └── BleDeviceRegistry.swift  
└── Devices/  
    ├── DeviceA/ (Device A: constants, models, protocol)  
    │   ├── DeviceAConstants.swift  
    │   ├── DeviceAModels.swift  
    │   └── DeviceAProtocol.swift  
    └── DeviceB/(optional; Device B UUIDs only)  
        └── DeviceBConstants.swift
```



RECOMMENDED ARCHITECTURE

Benefits:

- Loose coupling
- Easier debugging
- Testable
- Scales cleanly



04 RELIABILITY CHALLENGES

RANDOM DISCONNECTS

Apps must detect and recover from unexpected disconnects gracefully.

Common causes:

- Devices disconnect randomly
- Weak signals or interference
- Firmware bugs
- Background interruptions

RECONNECTION STRATEGY

Reliable apps automatically recover from disconnects.

Typical flow:

- Detect disconnect
- Wait a short delay
- Attempt reconnect
- Retry with limits

WRITE RELIABILITY ISSUES

BLE writes are **unreliable** without controlled message flow

BLE writes can fail due to:

- MTU size limits
- Packet collisions
- Device processing limits

COMMAND QUEUE PATTERN

Serializing commands prevents devices from being overwhelmed.

Flow:

Command queued

- Send command
- Wait for response
- Send next command

/// HANDLING FIRMWARE DIFFERENCES

Hardware evolves, so apps must support multiple firmware versions.

Challenges:

- Different firmware versions
- New features added
- Older devices still supported

Strategies:

- Firmware version check
- Feature flags
- Compatibility layers



05 TESTING & DEBUGGING TOOLS

▮ BLE DEBUGGING APPS

BLE debugging apps help isolate device problems quickly.

Tools:

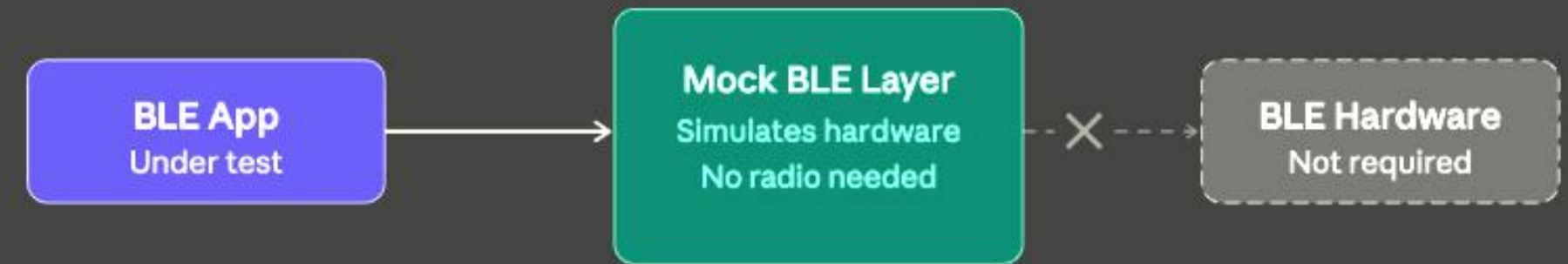
- LightBlue
- nRF Connect

MOCK BLE LAYER FOR TESTING

Mocks allow BLE apps to be tested without hardware.

Benefits:

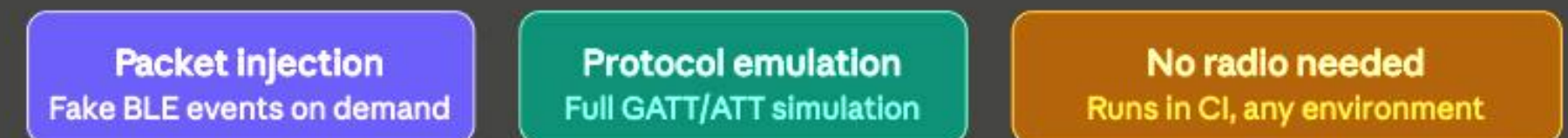
- UI testing without real devices
- Automated testing
- Faster development cycles



Mock packet exchange



Why mocks work



Combining logs, tools, and signal monitoring reveals most BLE issues.

Approaches:

- Monitor RSSI signal strength
- Track connection lifecycle
- Detect communication timeouts
- Inspect characteristic data



06

LIVE
DEMO

DEMO

Seeing the BLE lifecycle
in action makes the
concepts concrete.





07 PRODUCTION LESSONS

Design BLE apps assuming the connection will fail.

- Devices may behave unpredictably
- Firmware bugs are common
- Connections can drop unexpectedly
- Robust retry mechanisms are essential
- Strong logging and monitoring

Key Takeaways

Reliable BLE apps come from strong architecture and defensive design.

Reliable BLE apps require:

- Clear architecture
- State management
- Command queues
- Robust reconnection strategies
- Strong logging and monitoring



Pradip Sutariya

Associate Technical Lead,
Aubergine Solutions

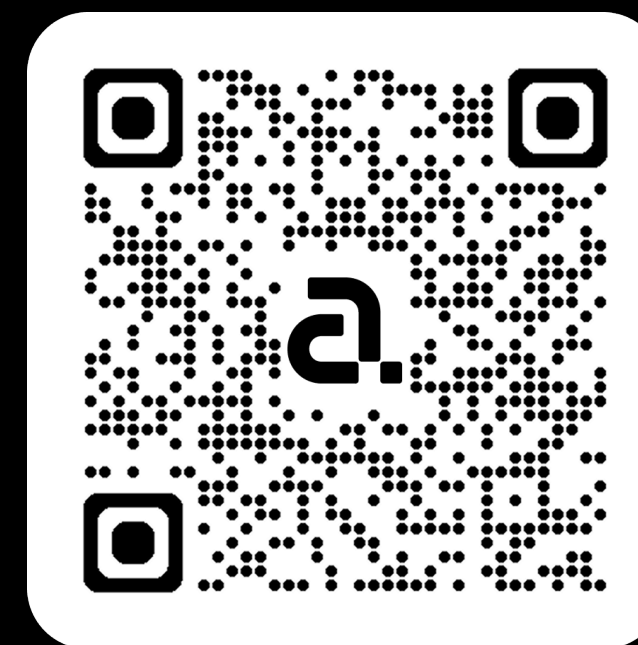
Scan QR and reach out
to me for venue inquiry



Rahul Parmar

Associate Software
Engineer,
Aubergine Solutions

Scan QR and reach out
to me for venue inquiry



aubergine.

Thank You!



Avinash Kariya

Software Engineer,
Aubergine Solutions

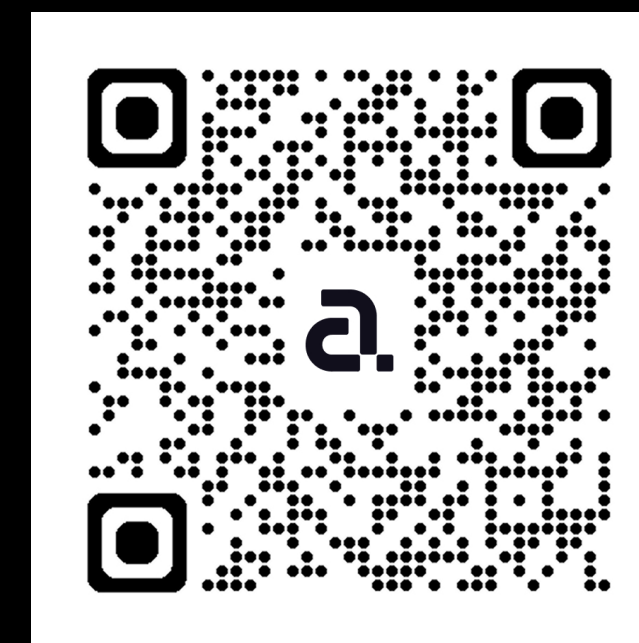
Scan QR and reach out
to me for venue inquiry



Parshwa Mehta

Software Engineer,
Aubergine Solutions

Scan QR and reach out
to me for venue inquiry



reach out to events@aubergine.co